Henry Shires

Section 3 Spring 2022

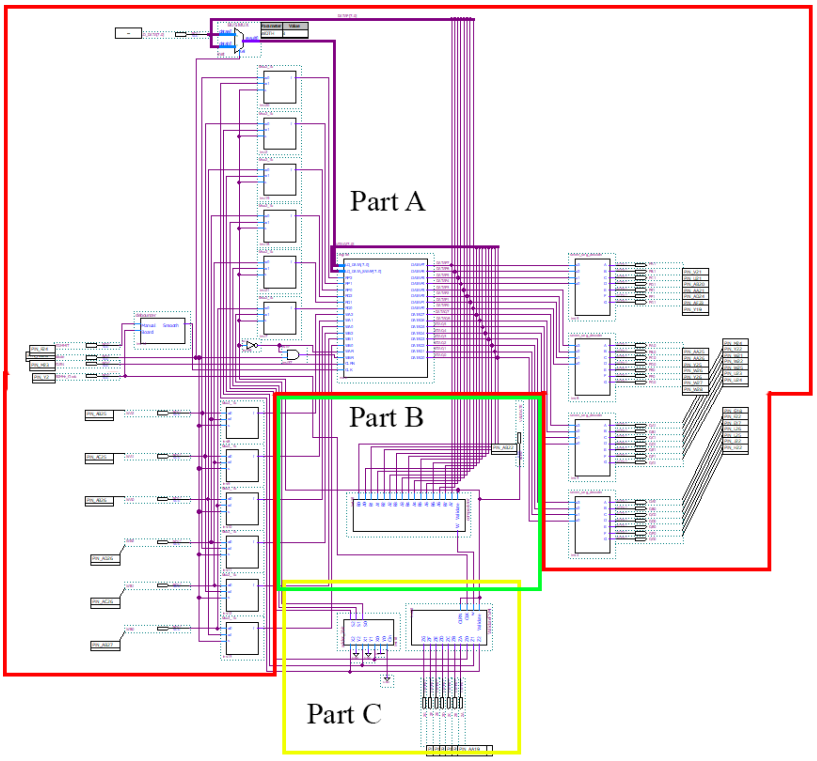
CPR E 281

Student ID: 398669315

Final Project Report – Manual Sorting Circuitry

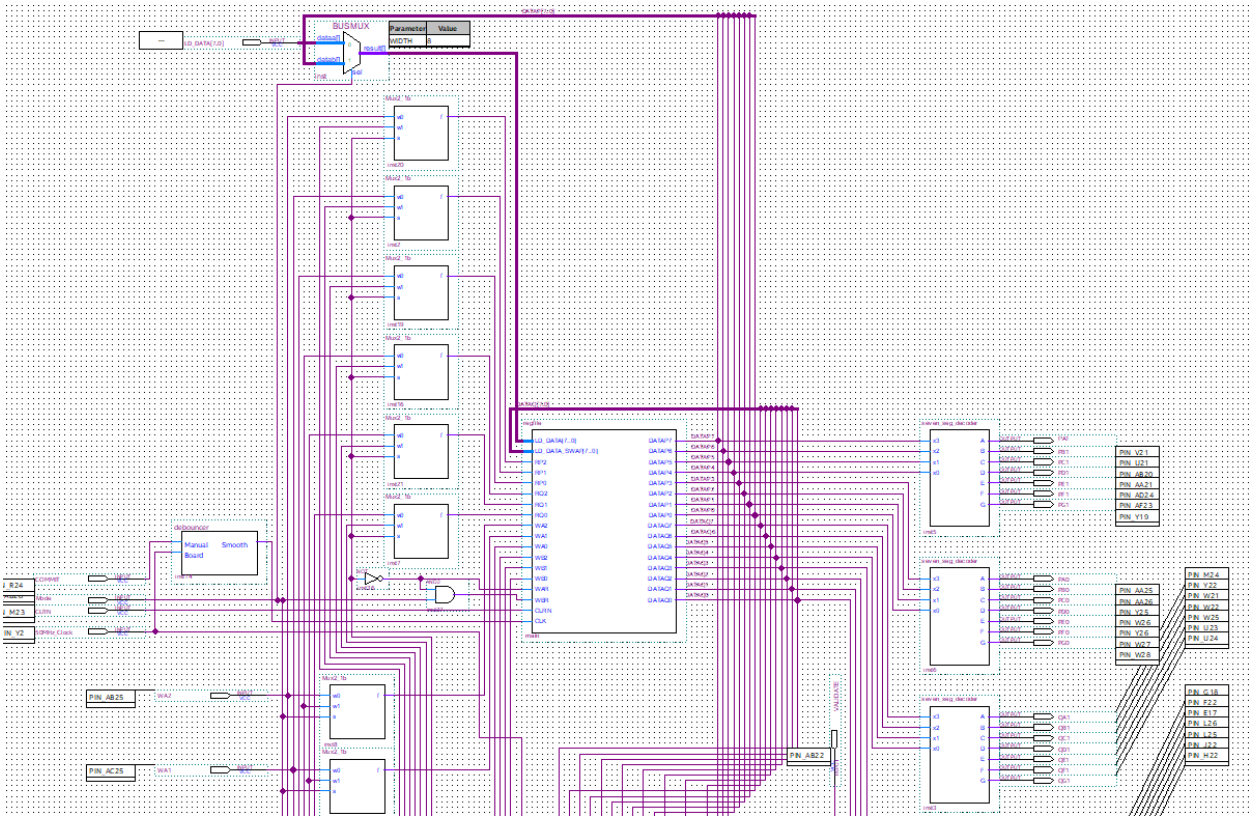
This report contains my development steps and components used to create the manual sorting circuitry described in project option one. A top-level diagram of my design is below, followed by a detailed examination of the sub-components that make up the design.

Top-Level Diagram



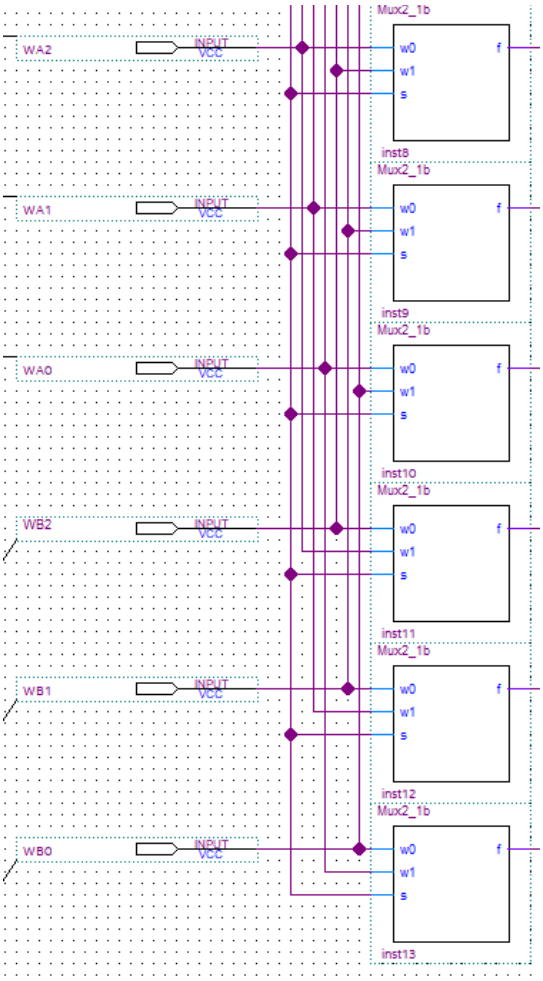
This diagram illustrates how each required section from the project rubric is connected and layered to create the final circuit. Parts A, B, and C represent the project's three main structures, where part A is the components for the register file, B is for the comparator, and C is for the state machine. Each section includes a detailed breakdown of their respective functionality.

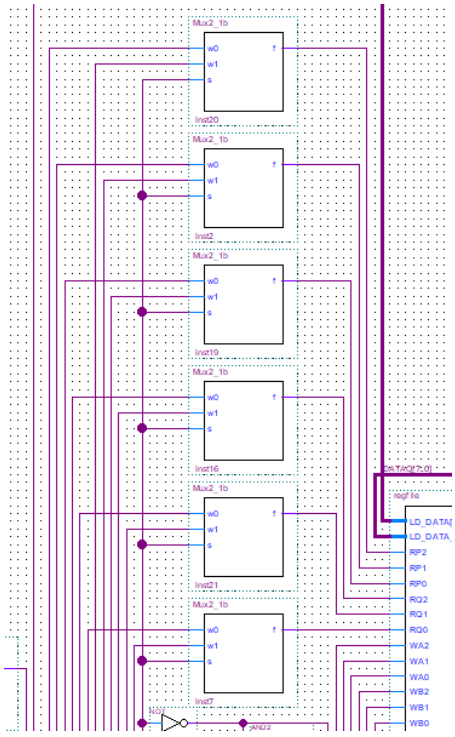
Part A – Register File & Seven-Segment Display Output



This section contains a register file with eight 8-bit registers to store numbers inputted that the user wishes to sort. The register file essentially works as a list. The *Mode* input act as the select bit for multiple

2-1 multiplexers (labeled **Mux2_1b***)* tasked with determining which data, read, and write ports have access to what registers in the register file. The corresponding switch for this input on the Altera board switches the register file between initialization and sorting modes. In initialization mode (the switch is 0), the register file allows one number to be written to one of its registers at a time using the *LD_DATA* port, an 8-bit bus input assigned to eight manual switches on the Altera board. The user directly inputs one address corresponding to a register to which they wish to write a number. Otherwise, when the switch is 1, the register file operates in sorting mode, allowing the user to swap the location positions of numbers previously entered in the register file. In sorting mode, the user inputs two addresses sent to the <u>read</u> ports *WA2, WA1, and WA0.* The output lines of the 2-1 MUXes connect directly to the decoder ports labeled accordingly for use in the register file explained later. Additional lines *WB2, WB1,* and *WB0* connect to other 2-1 MUXes (pictured in the original diagram) to utilize the second decoder port in the register file. This second set can either read the contents of a different register (initialization mode) or input the second of two addresses used to swap values (sorting mode). These write ports are also tied to corresponding read ports *RP…RQ*. Next, since the register file has two write ports, the register file must also have two read ports and an additional data port to enable manual swapping properly.





LD_DATA_SWAP is an additional 8-bit bus input primarily used for swapping two numbers. The user uses the Altera switches to set two 3-bit numbers into WA and WB decoders. The circuit uses inputs of the decoders first to read two existing numbers from the register file, then to write those numbers in *each other's* original locations (number 1 goes to register two, and number 2 goes to register 1). This operation happens simultaneously with the same clock pulse from the COMMIT switch, preventing the data from being overwritten. The read ports are later used in validation mode when VALIDATE = 1 and the comparator/finite state machine (FSM) combination requires automated reads from the register file (see Part B and C). Finally, two sets of two seven-segment displays and the DATAP and DATAQ sets of reading ports enable the user to view the register file's output. Outlined below are the components of the register file and the rest of Part A.

Mux2_1b

The **Mux2_1b** is simply an implementation of the BUSMUX for 1-bit wide wires. I realized later that it would be simpler to use the BUSMUX at a width of 1 anyway, but I chose to implement this as my work instead. Here's the Verilog code involving a simple conditional with a selector input, *s*.

```
1   module Mux2_1b(w0, w1, s, f);
2       input w0, w1, s;
3       output f;
4
5       assign f = s ? w1 : w0;
6   endmodule
```

Regfile

The register file implementation in Verilog is below. After completing lab 12, it was easier to expand upon the initial functionality of the register file in Verilog rather than revising it in block design. The register file is the most significant component in Part A and the entire top-level design. Many inputs include six 1-bit reads for two read ports total **(***RP, RQ***)**, six 1-bit writes for two write ports (*WA, WB***)**, the two data load ports 8-bit wide (*LD_DATA***,** *LD_DATA_SWAP***)**, two decoders (**Decoder3to8)**, two multiplexers (**Mux8_4b)**, and the eight 8-bit registers (**reg8b)**. To abide by two read ports, *DATAP[7..0]* and *DATAQ[7..0]* are used with the four seven-segment decoders pictured in Part A and explained below

to allow for swapping two 8-bit numbers simultaneously. Although I chose to make *DATAP* and *DATAQ* separate lines for each bit, it would've been simpler to design them as 8-bit buses.

Additionally, intermediate **wires** connect mentioned components similarly to how components connect in block design. Finally, in the registers *.IN()* and *.LOAD()* ports, I added conditional logic to choose based on the circuit mode whether to accept new data from LD_DATA or swap with LD_DATA_SWAP and LD_DATA together. When decoder WB is in use, LD_DATA_SWAP is read at that register to load two numbers to swap, and the load for each register is ORed with both decoders' load outputs so both decoders can control the register file.

```
module regfile(LD_DATA, LD_DATA_SWAP, DATAP7, DATAP6, DATAP5, DATAP4, DATAP3, DATAP2, DATAP1, DATAP0, DATAQ7, DATAQ6, DATAQ5, DATAQ4, DATAQ3, DATAQ2, DATAQ1, DATAQ0,
 RP2, RP1, RP0, RQ2, RQ1, RQ0, WA2, WA1, WA0, WB2, WB1, WB0, WAR, WBR, CLRN, CLK);

    // Address and control ports
    input RP2, RP1, RP0, RQ2, RQ1, RQ0, WA2, WA1, WA0, WB2, WB1, WB0, WAR, WBR, CLRN, CLK;

    // Input data port
    input [7:0] LD_DATA;
    input [7:0] LD_DATA_SWAP;

    // output data ports
    output DATAP7, DATAP6, DATAP5, DATAP4, DATAP3, DATAP2, DATAP1, DATAP0, DATAQ7, DATAQ6, DATAQ5, DATAQ4, DATAQ3, DATAQ2, DATAQ1, DATAQ0;

    // wire declarations
    wire X0, X1, X2, X3, X4, X5, X6, X7;
    wire Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7;
    wire [7:0] VALUE0, VALUE1, VALUE2, VALUE3, VALUE4, VALUE5, VALUE6, VALUE7;
    wire [7:0] DATAP, DATAQ;

    // Decoder to write to register file
    Decoder3to8 reg_decoder(.EN(WAR), .W2(WA2), .W1(WA1), .W0(WA0), .Y0(X0), .Y1(X1), .Y2(X2), .Y3(X3), .Y4(X4), .Y5(X5), .Y6(X6), .Y7(X7));
    Decoder3to8 reg_decoder2(.EN(WBR), .W2(WB2), .W1(WB1), .W0(WB0), .Y0(Y0), .Y1(Y1), .Y2(Y2), .Y3(Y3), .Y4(Y4), .Y5(Y5), .Y6(Y6), .Y7(Y7));

    // 8 8-bit registers
    reg8b reg0(.IN(Y0 ? LD_DATA_SWAP : LD_DATA), .LOAD(X0 | Y0), .CLK(CLK), .OUT(VALUE0), .CLRN(CLRN));
    reg8b reg1(.IN(Y1 ? LD_DATA_SWAP : LD_DATA), .LOAD(X1 | Y1), .CLK(CLK), .OUT(VALUE1), .CLRN(CLRN));
    reg8b reg2(.IN(Y2 ? LD_DATA_SWAP : LD_DATA), .LOAD(X2 | Y2), .CLK(CLK), .OUT(VALUE2), .CLRN(CLRN));
    reg8b reg3(.IN(Y3 ? LD_DATA_SWAP : LD_DATA), .LOAD(X3 | Y3), .CLK(CLK), .OUT(VALUE3), .CLRN(CLRN));
    reg8b reg4(.IN(Y4 ? LD_DATA_SWAP : LD_DATA), .LOAD(X4 | Y4), .CLK(CLK), .OUT(VALUE4), .CLRN(CLRN));
    reg8b reg5(.IN(Y5 ? LD_DATA_SWAP : LD_DATA), .LOAD(X5 | Y5), .CLK(CLK), .OUT(VALUE5), .CLRN(CLRN));
    reg8b reg6(.IN(Y6 ? LD_DATA_SWAP : LD_DATA), .LOAD(X6 | Y6), .CLK(CLK), .OUT(VALUE6), .CLRN(CLRN));
    reg8b reg7(.IN(Y7 ? LD_DATA_SWAP : LD_DATA), .LOAD(X7 | Y7), .CLK(CLK), .OUT(VALUE7), .CLRN(CLRN));

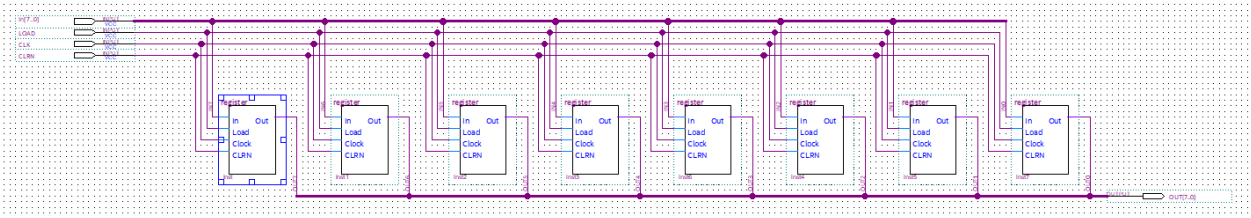
    // Read MUXs
    Mux8_4b muxP(.S2(RP2), .S1(RP1), .S0(RP0), .W0(VALUE0), .W1(VALUE1), .W2(VALUE2), .W3(VALUE3), .W4(VALUE4), .W5(VALUE5),.W6(VALUE6), .W7(VALUE7), .F(DATAP));
    Mux8_4b muxQ(.S2(RQ2), .S1(RQ1), .S0(RQ0), .W0(VALUE0), .W1(VALUE1), .W2(VALUE2), .W3(VALUE3), .W4(VALUE4), .W5(VALUE5),.W6(VALUE6), .W7(VALUE7), .F(DATAQ));

    assign DATAP7 = DATAP[7];
    assign DATAP6 = DATAP[6];
    assign DATAP5 = DATAP[5];
    assign DATAP4 = DATAP[4];
    assign DATAP3 = DATAP[3];
    assign DATAP2 = DATAP[2];
    assign DATAP1 = DATAP[1];
    assign DATAP0 = DATAP[0];
    assign DATAQ7 = DATAQ[7];
    assign DATAQ6 = DATAQ[6];
    assign DATAQ5 = DATAQ[5];
    assign DATAQ4 = DATAQ[4];
    assign DATAQ3 = DATAQ[3];
    assign DATAQ2 = DATAQ[2];
    assign DATAQ1 = DATAQ[1];
    assign DATAQ0 = DATAQ[0];

endmodule
```

Reg8b

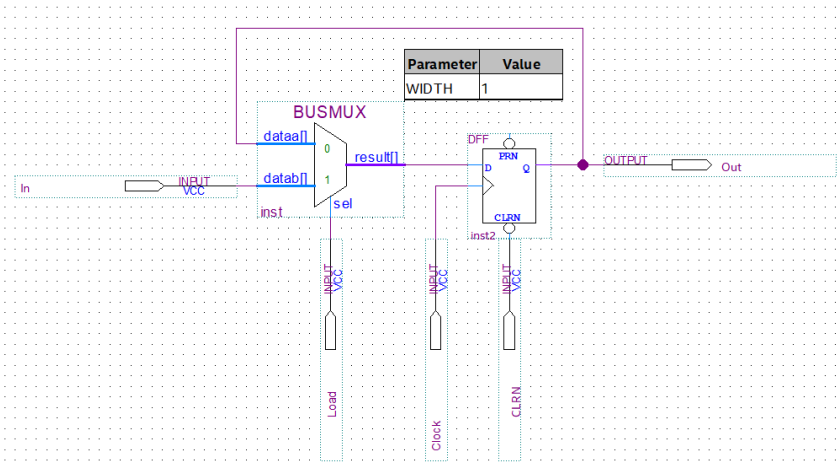
Reg8b is an 8-bit register, including an 8-bit input line, load line, clock, and CLRN. The register is parallel-load, allowing all eight outputs to work in parallel in the read ports. The block diagram for each 8-bit register is below (utilizing eight 1-bit register modules).



Register

An individual register bit is below, using a D Flip-Flop as a storage element. The input is either read if *load* = 1 or the current value of Q is latched if *load* = 0. The clock pulse is attached from the **reg8b** clock,

a manual pulse from the user to write on-demand. *CLRN* is used to force the output of $Q = 0$ when the user wishes to clear the register file.



Decoder3to8

Decoder3to8 is used in the two write ports of the register file to convert a 3-bit number representing an address + 1-bit enable to a one-hot encoded 8-bit address used to load data on a register with that corresponding address. This decoder utilizes a switch/case statement only with conditions where enable = 1 since the decoder is not in use when enable = 0. The Verilog implementation is below.

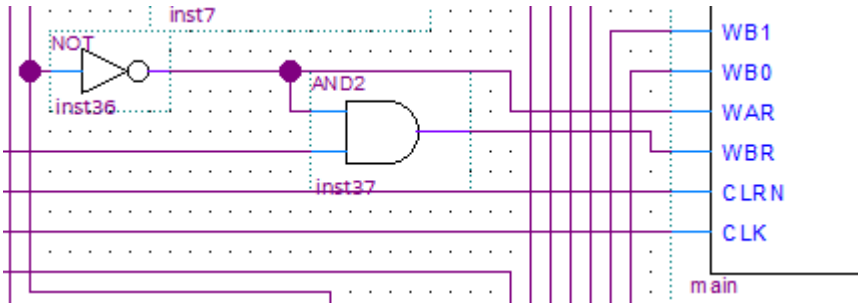
```
module Decoder3to8(EN, W2, W1, W0, Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7);
   input EN, W2, W1, W0;
   output reg Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7;

   always @(W2, W1, W0, EN)
   begin
      case({EN, W2, W1, W0})
         4'b1000: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b10000000;
         4'b1001: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b01000000;
         4'b1010: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b00100000;
         4'b1011: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b00010000;
         4'b1100: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b00001000;
         4'b1101: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b00000100;
         4'b1110: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b00000010;
         4'b1111: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b00000001;
         default: {Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7} = 8'b00000000;
      endcase
   end
endmodule
```

Write Port Enable

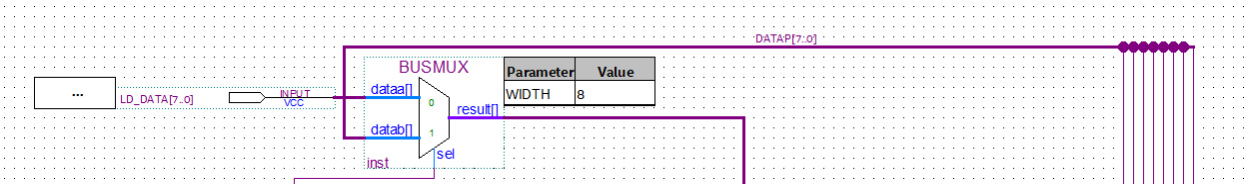
In my overall circuit, I implemented an extra precaution to disable write access to the register file when the following conditions are true. The user should be unable to write or swap numbers when the VALIDATE switch is set to 1 (the FSM is enabled), and the user should only be able to use decoder WB

when the register file is in sorting mode. Thus, I implemented the gates below to enable WA and WB decoders based on those cases (WBR = 1 when VALIDATE = 0 AND mode = 1, WAR when VALIDATE = 0).



LD_DATA Ports

The 2-1 BUS Multiplexer chooses whether to input the *LD_DATA* 8-bit bus data port directly into the register file (Initialization Mode) or to use the existing data in the register file to allow for manual swap. LD_DATA, regardless, applies to the WA decoder, while the second load port LD_DATA_SWAP only applies to the WB decoder and cannot input new data. It is only used for existing data to swap.



Mux8_4b

Mux8_4b uses eight inputs and three selectors to choose the register contents to output from the register file. One MUX can output one 8-bit number from one register, signifying one read port from the register file. To allow for two read ports, I implemented two MUXes total. The Verilog implementation uses the ternary or conditional operator "?" to simplify a conditional statement used to determine the output based on the selector values.

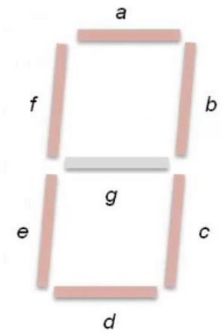
```
module Mux8_4b(S2, S1, S0, W0, W1, W2, W3, W4, W5, W6, W7, F);
   input S2, S1, S0;
   input [7:0] W0, W1, W2, W3, W4, W5, W6, W7;
   output [7:0] F;

   assign F = S2 ? (S1 ? (S0 ? W7 : W6) : (S0 ? W5 : W4)) : (S1 ? (S0 ? W3 : W2) : (S0 ? W1 : W0));

endmodule
```

Seven_seg_decoder

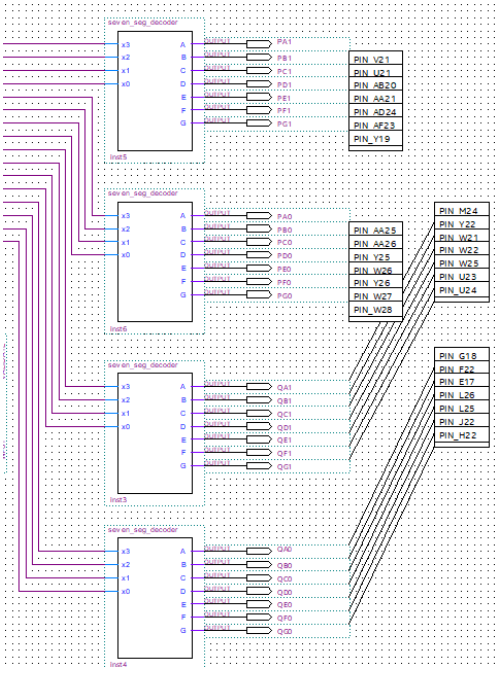
Seven_seg_decoder is a circuit designed to convert a 4-bit number to a 7-bit output compatible with the seven-segment display on the Altera board. The four inputs, with *x3* being the most significant bit of the input number, are then decoded into hexadecimal (NOT one-hot) to match the seven segments located on each display. A-F outputs in my Verilog implementation correspond to the segments in the diagram on the left. It utilizes a switch/case statement to set the output of all seven ports relatively easily:




```
module seven_seg_decoder(A, B, C, D, E, F, G, x3, x2, x1, x0);
   input x3, x2, x1, x0;
   output A, B, C, D, E, F, G;

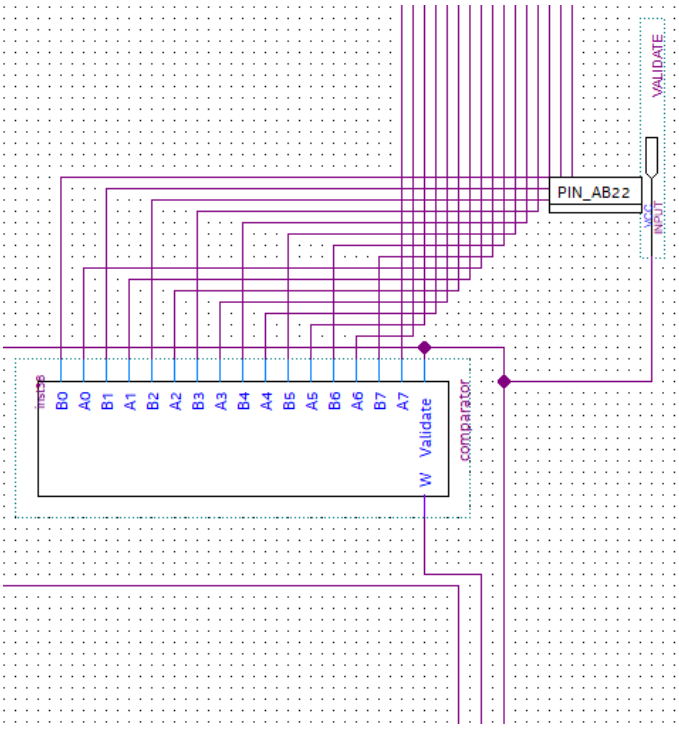
   reg A, B, C, D, E, F, G;

   always @(x3 or x2 or x1 or x0)
   begin
      case({x3, x2, x1, x0})
         4'b0000: {A, B, C, D, E, F, G} = 7'b0000001;
         4'b0001: {A, B, C, D, E, F, G} = 7'b1001111;
         4'b0010: {A, B, C, D, E, F, G} = 7'b0010010;
         4'b0011: {A, B, C, D, E, F, G} = 7'b0000110;
         4'b0100: {A, B, C, D, E, F, G} = 7'b1001100;
         4'b0101: {A, B, C, D, E, F, G} = 7'b0100100;
         4'b0110: {A, B, C, D, E, F, G} = 7'b0100000;
         4'b0111: {A, B, C, D, E, F, G} = 7'b0001111;
         4'b1000: {A, B, C, D, E, F, G} = 7'b0000000;
         4'b1001: {A, B, C, D, E, F, G} = 7'b0000100;
         4'b1010: {A, B, C, D, E, F, G} = 7'b0001000;
         4'b1011: {A, B, C, D, E, F, G} = 7'b1100000;
         4'b1100: {A, B, C, D, E, F, G} = 7'b0110001;
         4'b1101: {A, B, C, D, E, F, G} = 7'b1000010;
         4'b1110: {A, B, C, D, E, F, G} = 7'b0110000;
         4'b1111: {A, B, C, D, E, F, G} = 7'b0111000;
      endcase
   end
endmodule
```



Four decoders are used in the output of the register file to display two registers' contents at a time in a hexadecimal format based on the output of the number. Lines *DATAP* and *DATAQ* utilize two decoders each since both are 8-bit numbers.

Part B – 8-bit Comparator



When the user is finally ready to verify the sorted state of the register file, they may flip the *VALIDATE* switch, beginning the procedure to determine if the contents of the register file are in ascending order. Part B describes the first half of this procedure, where the machine determines if a number in one register is less than or greater than the number in the preceding register. For example, if the 8-bit number represented by inputs *A7-A0* are from the register with address 0x2 and *B7-B0* are inputs from register address 0x3, the value of *A[7..0]* must be less than *B[7..0]* to be considered sorted. Part C, discussed later, iterates through the entire register file until it compares all of the registers or detects an unsorted register. The **8bit_comparator** circuit pictured above executes this comparison step. In designing this component, I first brainstormed using the basic 1-bit comparator circuit discussed in a previous lecture. After analyzing, I noticed a pattern with the 1-bit, 2-bit, and 4-bit comparator versions based on the number of AND, XNOR, and NOT gates and how the inputs were stringed together. I noticed the greater than, equal to, and less than logic all had a partition of gates used to create each comparison for each bit. My derivation and initial design are pictured below.

Comparison Circuit - Number Comparator Doc #1 (Screenshot on Drive)

Is A >, <, or = B?

Inputs		Outputs		
A	B	A>B	A=B	A<B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0
		G	E	L

$G = A\bar{B}$

$\rightarrow E = \bar{A}\bar{B}$

$L = \bar{A}B$

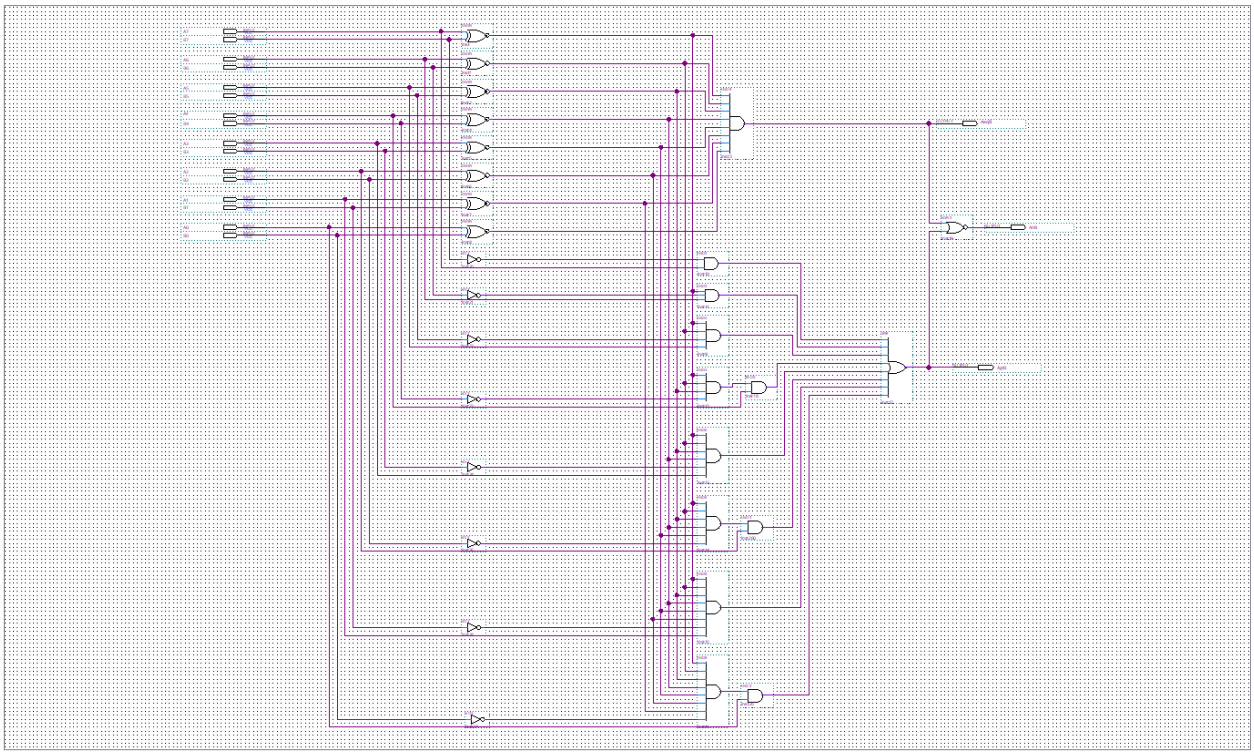
What about 2-bit? 4-bit? 8bit?

A_1A_0	B_1B_0	A<B	A=B	A>B
0 0	0 0	0	1	0
0 0	0 1	1	0	0
0 0	1 0	1	0	0
0 0	1 1	1	0	0
		0	0	1
1 1	1 1	0	1	0

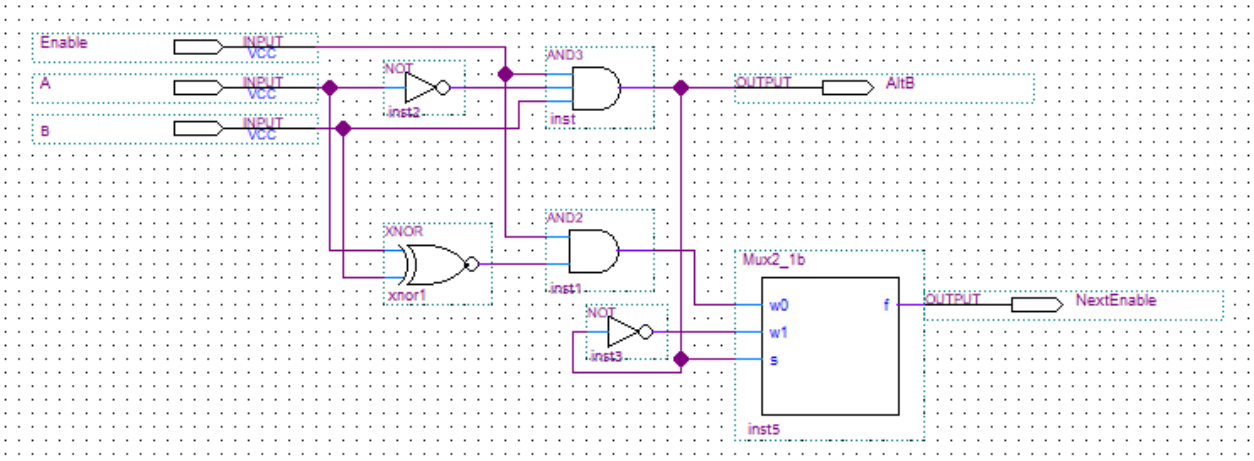
XNOR

Solutions: Combine NOT, OR, AND, based on 1-bit/2-bit comparator pattern

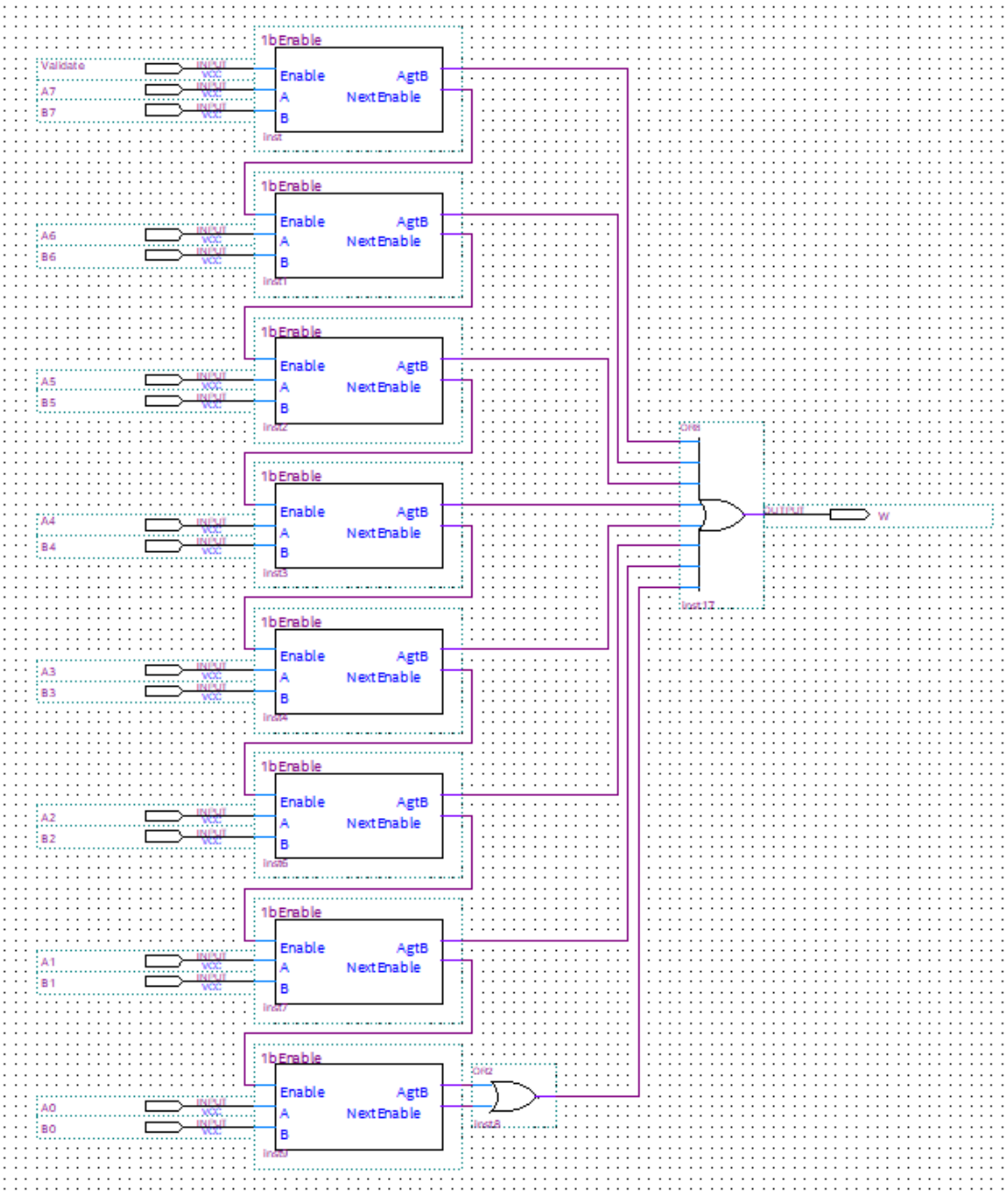
✓ N number of XNORs for N bits per number compared
✓ N ANDs for N bits
✓ All XNORs ANDed to output $A_{eq}B$ (A = B)
✓ All ANDs ORed to output $A_{gt}B$ (A > B) → Use NOT on one bit of one input number for each
✓ AeqB and AgtB NORed to output $A_{lt}B$ (A < B)



While this design worked as intended, I soon realized that it was not entirely compatible with the finite state machine when implementing it in the combined circuit (Part C). I needed a way to enable and disable the comparator so the FSM would "latch" its output (explained in Part C). But since my design above was large-scale and hard to revise, I had to rethink my strategy. Instead, I brainstormed and discovered the 8-bit comparison logic was simply eight 1-bit comparisons combined in either one AND gate (equal to output) or one OR gate (greater than output). In my revision, I instead used eight symbol blocks that contained the comparison logic and one large block to add an *ENABLE* line that enables or disables the next bit comparison block. This new layout creates the latching result intended for the final circuit. My revisions are pictured below with the single-bit comparison followed by the combined chain for an 8-bit input.

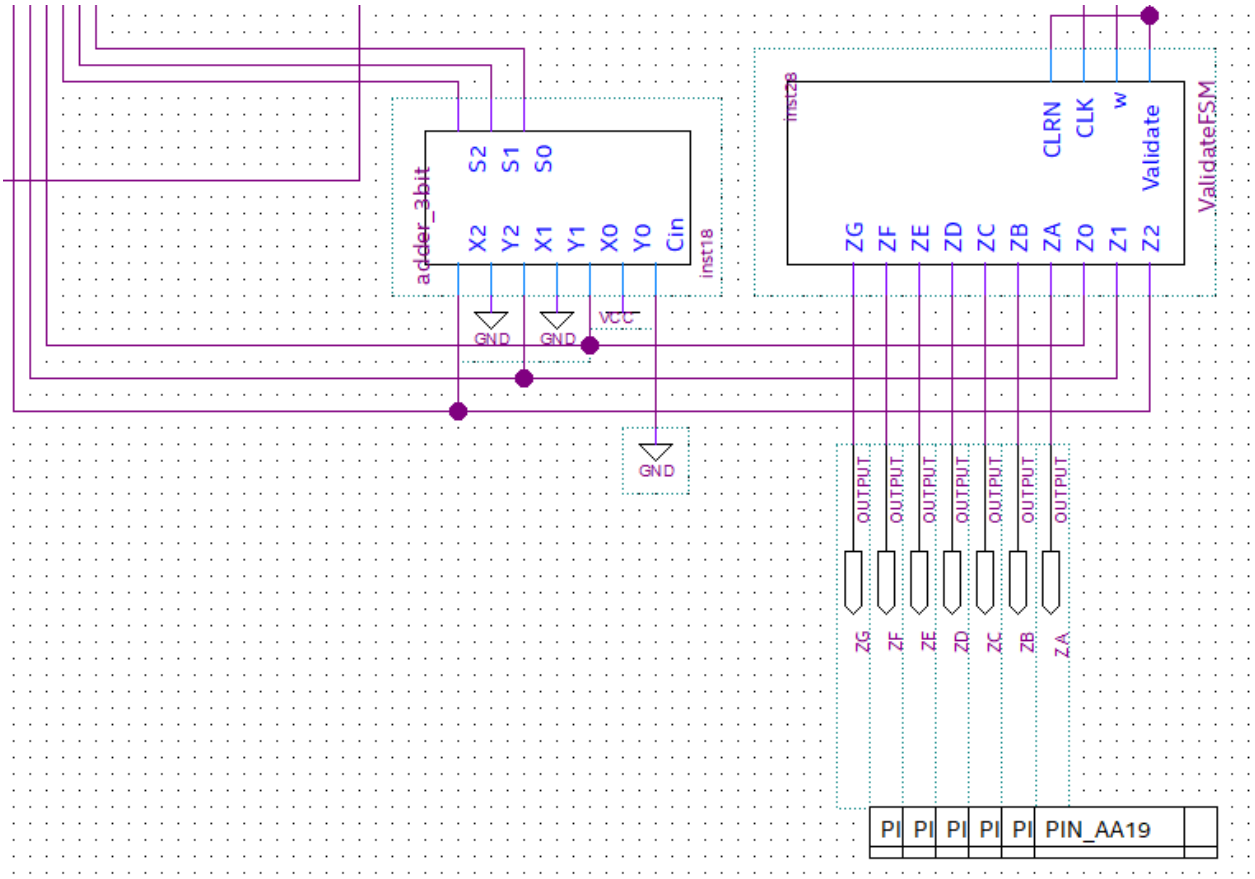


Two inputs, *A* and *B*, and an *Enable* input, are compared to be either less than, greater than, or equal. The less than logic expression requires *not(A)* ANDed with *B* as if *A* = 1 and *B* = 0, *not(A)* = 0, which is not less than *B*. But if *A* = 0 and *B* = 1, *not(A)* = 1, which evaluates the expression to 1, thus *A* is less than *B*. For equal to, *A* and *B* must be the same. Thus, an XOR with an inverter (XNOR) creates this check since this gate will evaluate to 1 when the two inputs are the same. Finally, for greater than, it is simply inverting the output of *AltB* (less than). In this circuit, the output *AltB* results from the less-than expression, and the *NextEnable* output enables or disables the succeeding 1-bit comparator in the chain circuit below. This output is determined by whether *A* is greater than *B* or not. The comparator detects an unsorted register when *A* is greater than B. This output means the register with value *A* has a larger value than its successor register with value *B*. Using a **Mux2_1b** as found previously in Part A, the output is selected by *AltB*, using either the output of *AequalsB* or *not(AltB)* as the output of *NextEnable*.



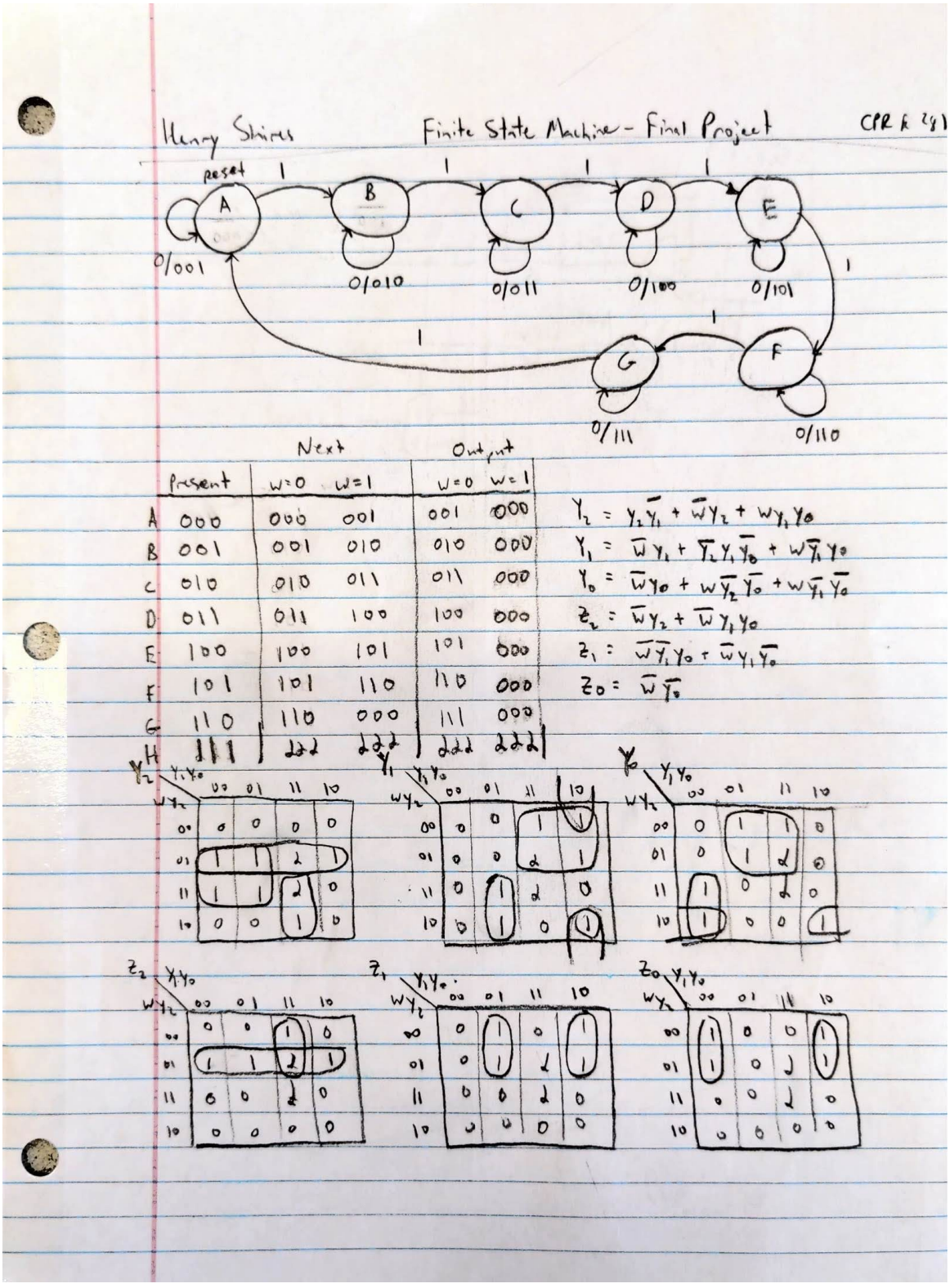
NextEnable links eight of the 1-bit comparators together and ORs the output of the final 1-bit comparator with *NextEnable* to "latch" the FSM if the FSM reaches the end of the register file or it detects an unsorted register. The FSM uses output *w* (Part C), which determines whether the FSM detected an unsorted register. If at least one of the bits is less than the bit at the register succeeding it, *w* is set to 1, allowing comparison of the following two registers as explained in Part C.

Part C – Finite State Machine (FSM)

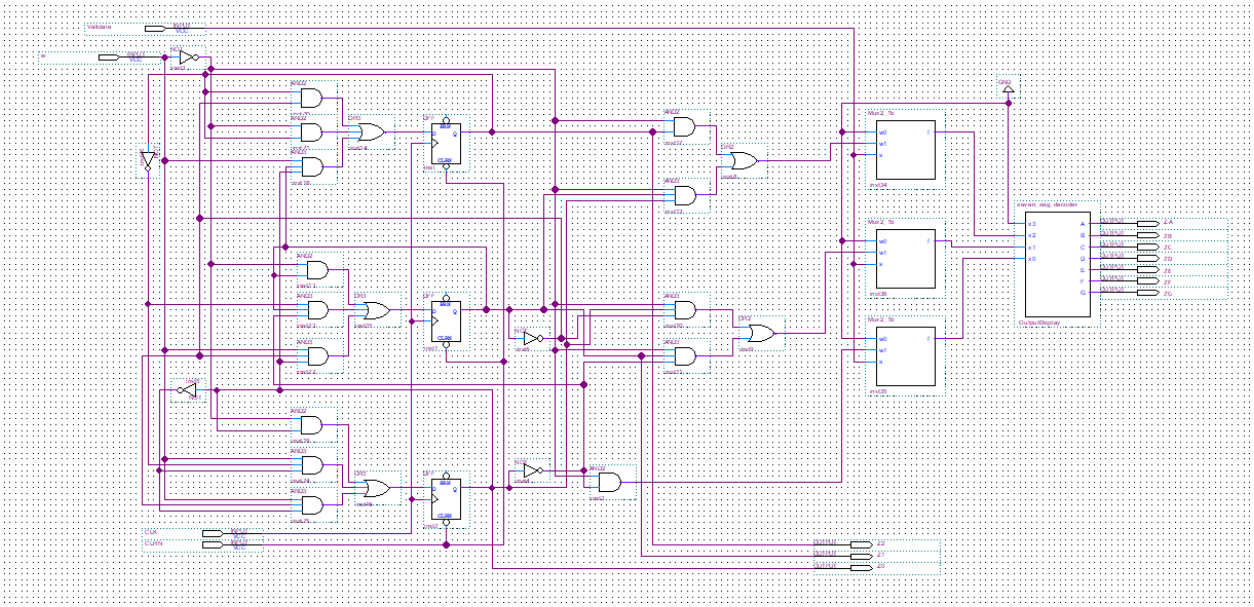


In the second half of the procedure, to determine the sorted state of the register file, Part C incorporates the Finite State Machine (FSM) used to iterate through the entire register file and use the comparator as its input. *W,* as described in Part B, is the output of the determination if two numbers from a register and its successor (the address + 1) are greater than, less than, or equal. *W* simplifies this computation to whether or not the two registers are in ascending order, the state that defines if the entire register file is sorted. I designed a Mealy-type machine, similar to the functionality of a counter, which moves to the next state if $w = 1$ (two registers are sorted) and latches on the present state if $w = 0$ (two registers are not sorted). If the registers are not sorted ($w = 0$), the FSM outputs the succeeding state; otherwise, the output is 0. The following image describes my derivation of the state machine, including the Mealy state diagram, state table, and K-maps to derive the logic expressions for the circuit diagram. I chose a Mealy

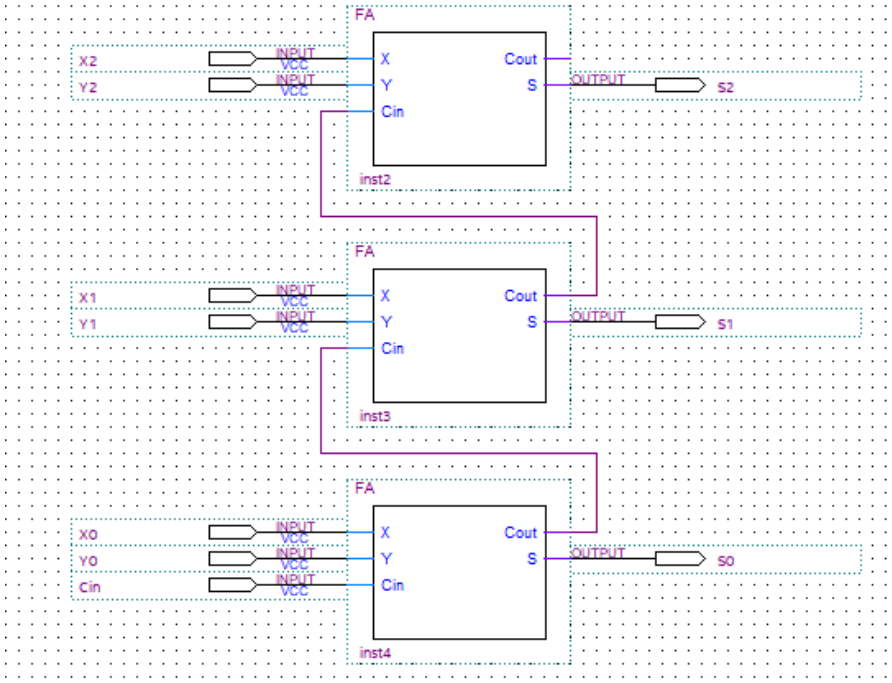
type, as the output relates to the state succeeding the present state, so it is more effective to have the machine depend on the input in addition to the state.



The input logic and D Flip-Flops use the expressions for *y*, while the output logic uses the expressions for *z*. However, my specific machine implements a seven-segment decoder as described in Part A, which uses the *z* output with the expressions in addition to the output from the D Flip Flops. The output of the seven-segment display (output from the decoder) is the register succeeding the machine's present state's corresponding register when $w = 0$. However, the FSM must also output the current state for its next iteration if $w = 1$ to use to check the rest of the register file. The FSM circuit diagram is implemented below, with combinational logic, DFFs, and output logic from left to right. Additionally, a 2-1 MUX uses VALIDATE as its selector bit to prevent instantaneous, erratic change in the output when $w = 1$ (hence matching the state table where the output should be 0).



This output returns to the register file, with the last three read lines running through a 3-bit adder **adder_3bit** to read the register succeeding the address of the register outputted by the FSM.



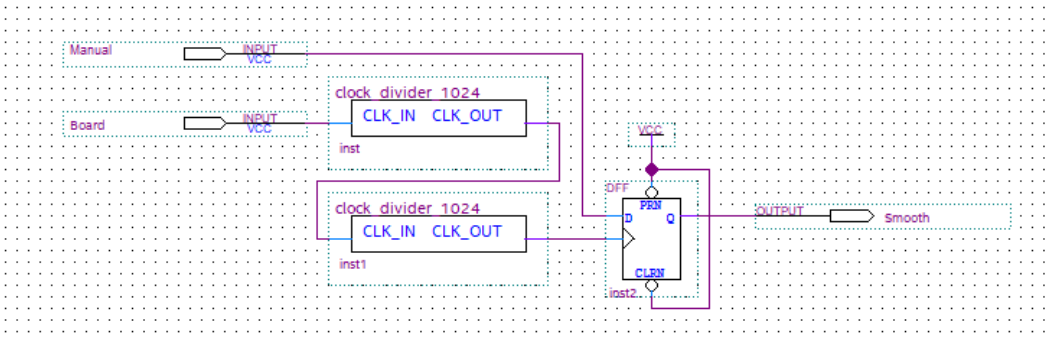
Three full adders **(FA)** are used to add 001 (1) to the 3-bit register address, which the register file later uses. The Verilog code for the fuller adder is below:

```
1   module FA(X, Y, Cin, Cout, S);
2      input Cin, X, Y;
3      output Cout, S;
4
5      assign S = X ^ Y ^ Cin;
6      assign Cout = (Cin & (X ^ Y)) | (X & Y);
7
8   endmodule
```

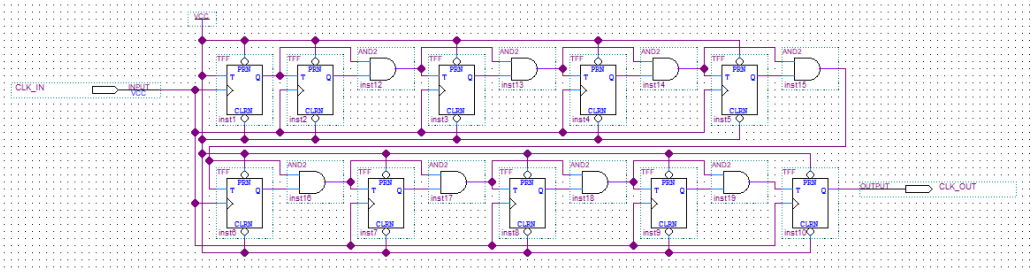
The output of the FSM, when it locates an unsorted register, is sent to the seven-segment display where the user is alerted of the unsorted register address. The user can turn off VALIDATE and choose to swap or write new numbers and attempt a new validation. During a new validation, The FSM first gets cleared when VALIDATE asserts to 1, then is re-validated either with a new register that is unsorted or no input if sorted.

Addendum: Debouncing the COMMIT Button

On the Altera board, the switch used for the COMMIT button (KEY3 pin R24) suffers from bouncing issues. This bug results in unintentional and inaccurate writes to the register file, causing potential overwrites and mismatched inputs. The following debounce module attached to the final design elevated this issue entirely, utilizing the onboard clock (pin Y2) to "smooth" out the button's input to prolong the state of the button transitioning from 0 to 1 and vice versa. See below:



The effect of the **Clock_Divider_1024** divides the clock frequency from the board twice by 1024 and synchronizes it with the manual input from the button. This division forces the signal to an output of 1 for an extended period to fix the abovementioned issues. The clock divider circuit is pictured below:



The debounce module is pictured attached to the COMMIT input and clock in the final design in Part A.

